CS Exebit 2026

Lean for Verification

Pranav Ramesh

Proof Assistants

What are proof assistants (interactive theorem provers)?

- Check and help develop formal proofs
- Can prove big theorems, not just logic puzzles
- Can be tedious to use
- Highly addictive (think video games!)

Classification by Logical Foundations

Set Theory:

- Isabelle/ZF
- Metamath
- Mizar

Dependent Type Theory:

- Agda
- **Lean** $\leftarrow$ We focus here
- Matita
- Rocq

Simple Type Theory:

- HOL4
- HOL Light
- Isabelle/HOL
- PVS

Success Stories in Formal Verification

Mathematics:

- The four-color theorem
- The Kepler conjecture
- The definition of perfectoid spaces

Computer Science:

- Hardware verification
- Operating systems
- Programming language theory
- Compilers
- Security

What is Lean?

- Proof assistant developed by Leonardo de Moura (AWS) since 2012
- Mathematical library `mathlib` developed by large community
- Community version of Lean 4
- Built on the **calculus of inductive constructions**

Key Features:

- Highly expressive dependent type theory
- Extended with classical axioms and quotient types
- Modern metaprogramming framework
- Modern user interface
- Open source with active community

A View of Lean

$$\text{Lean} = \text{Functional Programming} + \text{Logic}$$

Today's focus:

- Syntax of types and terms
- Basics of Functional Programming
- Proof Automation and metaprogramming

Types

Types α, τ, ν can be:

- Type variables: α
- Basic types: T
- Complex types: $T\,\sigma_1\,\ldots\,\sigma_N$

Function Types:

- Some type constructors written infix (e.g., $\rightarrow$)
- Right-associative: $\sigma_1 \rightarrow \sigma_2 \rightarrow \sigma_3 \rightarrow \tau = \sigma_1 \rightarrow (\sigma_2 \rightarrow (\sigma_3 \rightarrow \tau))$

Polymorphism:

- Type variables must be bound using $\forall$
- Example: $\forall\alpha,\, \alpha \rightarrow \alpha$

Terms and Applications

Terms t, u:

- Constants c
- Variables x
- Applications $t\,u$
- Anonymous functions $\lambda x \mapsto t$

Currying: Functions can be:

- Fully applied: $f\,x\,y\,z$
- Partially applied: $f\,x\,y$ or $f\,x$
- Unapplied: f

Application is left-associative: $f\,x\,y\,z = ((f\,x)\,y)\,z$

Type Checking and Inference

Key Facts:

- Both type checking and type inference are decidable
- This property is lost with overloading or subtyping

Type Judgment: $\Gamma \vdash t : \sigma$
Means: t has type σ in local context Γ

Typing Rules

Constant:

$$\frac{}{\Gamma \vdash c : \sigma} \quad \text{(if } c \text{ declared with type } \sigma\text{)}$$

Variable:

$$\frac{}{\Gamma \vdash x : \sigma} \quad \text{(if } x : \sigma \text{ in } \Gamma\text{)}$$

Application:

$$\frac{\Gamma \vdash t : \sigma \to \tau \quad \Gamma \vdash u : \sigma}{\Gamma \vdash t\, u : \tau}$$

Abstraction:

$$\frac{\Gamma, x : \sigma \vdash t : \tau}{\Gamma \vdash (\lambda x : \sigma \mapsto t) : \sigma \to \tau}$$

Imperative Factorial

```
factorial(n) {
  a = 1;
  while (n > 0) {
    a = a * n;
    n = n - 1;
  }
  return a;
}
```

Transition System for Factorial

$$\begin{array}{rcll} \text{Natural numbers} & n & \in & \mathbb{N} \\ \text{States} & s & ::= & \mathsf{AnswerIs}(n) \mid \mathsf{WithAccumulator}(n, n) \end{array}$$

$$\frac{}{\mathsf{WithAccumulator}(n_0, 1) \in \mathcal{F}_0} \qquad \frac{}{\mathsf{AnswerIs}(a) \in \mathcal{F}_\omega}$$

$$\frac{}{\mathsf{WithAccumulator}(0, a) \to \mathsf{AnswerIs}(a)}$$

$$\frac{}{\mathsf{WithAccumulator}(n+1, a) \to \mathsf{WithAccumulator}(n, a \times (n+1))}$$

Definition 5.1. A *transition system* is a triple $\langle S, S_0, \to \rangle$, with S a set of states, $S_0 \subseteq S$ a set of initial states, and $\to \subseteq S \times S$ a transition relation.

Correctness of factorial

- Transitive Reflexive Closure (trc)

$$\frac{}{s \rightarrow^* s} \qquad \frac{s \rightarrow s' \quad s' \rightarrow^* s''}{s \rightarrow^* s''}$$

- Reachable states of a transition system

DEFINITION 5.2. For transition system $\langle S, S_0, \rightarrow \rangle$, we say that a state s is *reachable* if and only if there exists $s_0 \in S_0$ such that $s_0 \rightarrow^* s$.

- Correctness of factorial encoded as transition system

For any state s reachable in $\mathcal{F}$, if $s \in \mathcal{F}_\omega$, then $s =$ AnswerIs$(n_0!)$.

Invariants

For any state s reachable in $\mathcal{F}$, if $s \in \mathcal{F}_\omega$, then $s =$ AnswerIs$(n_0!)$.

- We could prove this in an ad-hoc way. But let's develop a general notion of *invariants*
  - Invariant is a property of a programs that *starts true and stays true*

DEFINITION 5.4. An *invariant* of a transition system is a property that is always true, in all of the system's reachable states. That is, for transition system $\langle S, S_0, \rightarrow \rangle$, where R is the set of all its reachable states, some $I \subseteq S$ is an invariant iff $R \subseteq I$.

- Mathematically, *Property of states = Set of states*
  - The "property" holds of exactly those states that belong to the set

Set Representation

Define a set as a Boolean function:

```
def set (a:Type) [DecidableEq a] := a      Bool
```

Key Idea: A set is represented as a predicate that returns `true` or `false` for membership.

Basic Set Operations

Empty set:

```
def empty {a:Type} [DecidableEq a] : set a
  := fun x => false
```

Singleton set:

```
def singleton {a:Type} [DecidableEq a] (x: a): set a
  := fun y => y = x
```

Membership:

```
def mem {a:Type} [DecidableEq a] (x: a) (s: set a) :
   Bool
  := s x
```

Set Operations: Union & Intersection

Union:

```
def union {a:Type} [DecidableEq a] (s1 s2 : set a) :
    set a :=
  (fun x => s1 x || s2 x)
```

Intersection:

```
def intersection {a:Type} [DecidableEq a] (s1 s2: set
    a) : set a :=
  (fun x => s1 x && s2 x)
```

Set Operations: Complement & Difference

Complement:

```
def complement {a:Type} [DecidableEq a] (s : set a) :
    set a :=
  (fun x => not (s x))
```

Difference:

```
def difference {a:Type} [DecidableEq a] (s1 s2: set a)
     : set a :=
  (fun x => s1 x && not (s2 x))
```

Set Operations: Filter, Remove, Add

Filter by predicate:

```
def filter {a:Type} [DecidableEq a] (s1: set a) (p: a
        Bool) : set a :=
  (fun x => s1 x && p x)
```

Remove element:

```
def remove {a: Type} [DecidableEq a] (s1: set a) x :=
  (fun y => s1 y && x != y)
```

Add element:

```
def add {a: Type} [DecidableEq a] (x:a) (s: set a) :
    set a :=
  union s (singleton x)
```

Set Equality & Subset

Set Equality:

```
def equal {a:Type} [DecidableEq a] (s1: set a) (s2:
    set a) :=
       x : a, s1 x == s2 x
```

Subset relation:

```
def subset {a: Type} [DecidableEq a] (s1 s2: set a) :=
  forall x, mem x s1     mem x s2
```

Correctness Lemmas: Membership

Empty set has no members:

```
lemma mem_empty {a: Type} [DecidableEq a] (x: a) :
  not (mem x empty)
```

Membership in singleton:

```
lemma mem_singleton {a: Type} [DecidableEq a] (x y : a
    ):
  mem y (singleton x) = (x = y)
```

Correctness Lemmas: Union & Intersection

Union membership:

```
lemma mem_union {a: Type} [DecidableEq a] (s1 s2: set
    a) (x:a) :
  mem x (union s1 s2)      (mem x s1 || mem x s2)
```

Intersection membership:

```
lemma mem_intersection {a: Type} [DecidableEq a]
     (s1 s2: set a) (x: a) :
  mem x (intersection s1 s2)      (mem x s1      mem x
      s2)
```

Correctness Lemmas: Complement & Difference

Complement membership:

```
lemma mem_complement {a: Type} [DecidableEq a] (s: set
    a) (x: a) :
  mem x (complement s)      not (mem x s)
```

Difference membership:

```
lemma mem_difference {a: Type} [DecidableEq a]
    (s1 s2: set a) (x: a) :
  mem x (difference s1 s2)      (mem x s1          (mem x
      s2))
```

Correctness Lemmas: Filter & Remove

Filter membership:

```
lemma mem_filter {a: Type} [DecidableEq a]
    (s: set a) (p: a      Bool) (x: a):
  mem x (filter s p)      (mem x s      p x)
```

Remove element lemmas:

```
lemma mem_remove_x {a: Type} [DecidableEq a] (s: set a
   ) (x: a):
  not (mem x (remove s x))

lemma mem_remove_y {a: Type} [DecidableEq a]
    (s: set a) (x: a) (y: a) :
  x != y      mem y (remove s x) == mem y s
```

Summary: Custom Set Implementation

What we've built:

- Functional representation of sets using Boolean predicates
- Core operations: union, intersection, complement, difference
- Filtering, removal, and addition operations
- Formal proofs that all operations behave correctly

Key Benefits:

- Decidable equality constraints ensure computational feasibility
- All lemmas formally verified in Lean
- Any program using these operations is guaranteed correct